数值算法 Homework 10

姓名: 雍崔扬
学号: 21307140051
Due: 08:00 Apr. 27, 2025

Problem 1

Use a linear combination of $f(x)$, $f(x+h)$, $f(x+2h)$ to approximate $f'(x)$ as accurately as you can.
Give an error estimate by taking into account both truncation and rounding errors.

Solution:
设 f 在 x 附近充分光滑，则对于足够小的 $h>0$，我们有:

$$\begin{aligned}
f(x+h) &= \sum_{k=0}^{\infty} \frac{f^{(k)}(x)}{k!} h^k \\
&= f(x) + f'(x)h + \frac{f''(x)}{2!}h^2 + \frac{f^{(3)}(x)}{3!}h^3 + O(h^4) \\
f(x+2h) &= \sum_{k=0}^{\infty} \frac{f^{(k)}(x)}{k!} (2h)^k \\
&= f(x) + 2f'(x)h + 2f''(x)h^2 + \frac{4}{3}f^{(3)}(x)h^3 + O(h^4)
\end{aligned}$$

考虑线性组合 $af(x)+bf(x+h)+cf(x+2h)$:

$$\begin{aligned}
&af(x)+bf(x+h)+cf(x+2h) \\
&= af(x) + b\left[f(x) + f'(x)h + \frac{f''(x)}{2!}h^2 + \frac{f^{(3)}(x)}{3!}h^3\right] + c\left[f(x) + 2f'(x)h + 2f''(x)h^2 + \frac{4}{3}f^{(3)}(x)h^3\right] + O(h^4) \\
&= (a+b+c)f(x) + (b+2c)f'(x)h + \left(\frac{b}{2}+2c\right)f''(x)h^2 + \left(\frac{b}{6}+\frac{4c}{3}\right)f^{(3)}(x)h^3 + O(h^4)
\end{aligned}$$

解下列方程组:

$$\begin{cases} a+b+c=0 \\ b+2c=1 \\ \frac{b}{2}+2c=0 \end{cases} \Rightarrow \begin{cases} a=-\frac{3}{2} \\ b=2 \\ c=-\frac{1}{2} \end{cases}$$

因此我们有:

$$\begin{aligned}
-\frac{3}{2}f(x) + 2f(x+h) - \frac{1}{2}f(x+2h) &= f'(x)h - \frac{1}{3}f^{(3)}(x)h^3 + O(h^4) \\
&= f'(x)h - \frac{1}{3}f^{(3)}(\xi)h^3 \text{ for some } \xi \in [x, x+2h]
\end{aligned}$$

于是我们有:

$$\begin{aligned}
f'(x) &= \frac{-3f(x)+4f(x+h)-f(x+2h)}{2h} + \frac{1}{3}f^{(3)}(\xi)h^2 \text{ for some } \xi \in [x, x+2h] \\
&= \frac{-3f(x)+4f(x+h)-f(x+2h)}{2h} + O(h^2)
\end{aligned}$$

现考虑舍入误差 $\varepsilon := [-3f(x)+4f(x+h)-f(x+2h)] - \mathrm{fl}(-3f(x)+4f(x+h)-f(x+2h))$
则我们有:

$$f'(x) = \frac{-3f(x)+4f(x+h)-f(x+2h)}{2h} + O\left(h^2 + \frac{\varepsilon}{h}\right)$$

考虑最小化 $E(h)=h^2+\frac{\varepsilon}{h}$，便可知最优步长 $h_{\text{opt}}=\Theta(\varepsilon^{1/3})$，极限精度 $E_{\text{opt}}=\Theta(\varepsilon^{2/3})$

Problem 2

Use a linear combination ofF function values $f(x+ih,y+jh)$ (for $i,j\in\{-1,0,1\}$)
to approximate $\frac{\partial^2}{\partial x^2}f(x,y)+\frac{\partial^2}{\partial y^2}f(x,y)$ as accurately as you can.
Estimate the truncation error.

Solution: (reference)
各点函数值的 Taylor 展开:

- **边点 (Manhattan 距离为 1)**

$$
\begin{aligned}
f(x\pm h,y)&=f\pm hf_x+\frac{h^2}{2}f_{xx}\pm\frac{h^3}{6}f_{xxx}+\frac{h^4}{24}f_{xxxx}+O(h^5)\\
f(x,y\pm)&=f\pm hf_y+\frac{h^2}{2}f_{yy}\pm\frac{h^3}{6}f_{yyy}+\frac{h^4}{24}f_{yyyy}+O(h^5)
\end{aligned}
$$

  其中 f 代表 $f(x,y)$，f_x 代表 $\frac{\partial}{\partial x}f(x,y)$，$f_{xx}$ 代表 $\frac{\partial^2}{\partial x^2}f(x,y)$

- **角点 (Manhattan 距离为 2)**

$$
\begin{aligned}
f(x\pm h,y+h)=f&+h(\pm f_x+f_y)+\frac{h^2}{2}(f_{xx}\pm 2f_{xy}+f_{yy})\\
&+\frac{h^3}{6}(\pm f_{xxx}+3f_{xxy}\pm 3f_{xyy}+f_{yyy})\\
&+\frac{h^4}{24}(f_{xxxx}\pm 4f_{xxxy}+6f_{xxyy}\pm 4f_{xyyy}+f_{yyyy})\\
&+O(h^5)\\
\hline
f(x\pm h,y-h)=f&+h(\pm f_x-f_y)+\frac{h^2}{2}(f_{xx}\mp 2f_{xy}+f_{yy})\\
&+\frac{h^3}{6}(\pm f_{xxx}-3f_{xxy}\pm 3f_{xyy}-f_{yyy})\\
&+\frac{h^4}{24}(f_{xxxx}\mp 4f_{xxxy}+6f_{xxyy}\mp 4f_{xyyy}+f_{yyyy})\\
&+O(h^5)
\end{aligned}
$$

根据 Laplace 算子 $\frac{\partial^2}{\partial x^2}f(x,y)+\frac{\partial^2}{\partial y^2}f(x,y)$ 的旋转对称性，我们合理假设所需的线性组合形如:

$$
\begin{aligned}
L_h[f]&:=\alpha\sum_{|i|+|j|=1}f(x+ih,y+jh)+\beta\sum_{|i|+|j|=2}f(x+ih,y+jh)+\gamma f(x,y)\\
&=\alpha(f_{0,-1}+f_{0,1}+f_{-1,0}+f_{1,0})+\beta(f_{-1,-1}+f_{-1,1}+f_{1,-1}+f_{1,1})+\gamma f_{0,0}\\
&=\alpha\left(4f+h^2(f_{xx}+f_{yy})+\frac{h^4}{12}(f_{xxxx}+f_{yyyy})\right)\\
&\quad+\beta\left(4f+2h^2(f_{xx}+f_{yy})+\frac{h^4}{6}(f_{xxxx}+6f_{xxyy}+f_{yyyy})\right)\\
&\quad+\gamma f+O(h^5)\\
&=(4\alpha+4\beta+\gamma)+(\alpha+2\beta)(f_{xx}+f_{yy})h^2+\left[\left(\frac{\alpha}{12}+\frac{\beta}{6}\right)(f_{xxxx}+f_{yyyy})+\beta f_{xxyy}\right]h^4+O(h^5)
\end{aligned}
$$

其中 $f_{i,j}:=f(x+ih,y+jh)$ for $i,j\in\{-1,0,1\}$
求解以下方程组:

$$\begin{cases} 4\alpha+4\beta+\gamma=0 \\ \alpha+2\beta=1 \\ 2(\frac{\alpha}{12}+\frac{\beta}{6})=\beta \end{cases} \Rightarrow \begin{cases} \alpha=\frac{2}{3} \\ \beta=\frac{1}{6} \\ \gamma=-\frac{10}{3} \end{cases}$$

于是我们有:

$$\begin{aligned} & f_{xx}+f_{yy} \\ &= \frac{2}{3h^2}\sum_{|i|+|j|=1} f(x+ih,y+jh)+\frac{1}{6h^2}\sum_{|i|+|j|=2} f(x+ih,y+jh)-\frac{10}{3h^2}f(x,y)-\frac{1}{12}(f_{xxxx}+2f_{xxyy}+f_{yyyy})h^2+O(h^3) \\ &= \frac{4(f_{0,-1}+f_{0,1}+f_{-1,0}+f_{1,0})+1(f_{-1,-1}+f_{-1,1}+f_{1,-1}+f_{1,1})-20f_{0,0}}{6h^2}-\frac{1}{12}(f_{xxxx}+2f_{xxyy}+f_{yyyy})h^2+O(h^3) \\ &= \frac{4(f_{0,-1}+f_{0,1}+f_{-1,0}+f_{1,0})+1(f_{-1,-1}+f_{-1,1}+f_{1,-1}+f_{1,1})-20f_{0,0}}{6h^2}+O(h^2) \end{aligned}$$

Problem 3

Use Richardson extrapolation to estimate the derivative of $f(x)=x^3e^x$ at $x=1$.
Keep a record for intermediate results.
What happens if you iterate for many steps?

(1) Richardson 外推

Richardson 外推是提供误差逼近阶的重要手段，它通过组合一些较差的估计来得到更好的估计.
设 a 是未知量，而 $A(h)$ 是 a 的一个依赖于 h 的近似，且满足 $\lim_{h\to 0}A(h)=a$
假设 $A(h)$ 可以展开成如下形式:

$$a=A(h)+\sum_{i=k}^{m}a_ih^i+c_m(h)h^{m+1}$$

其中 $c_m(h)$ 是 h 的函数，a_i 是不依赖于 h 的常量，且 $a_k\neq 0$.
Richardson 外推的基本思想是先计算 a 的两个不同近似 $A(h_1)$ 和 $A(h_2)$，
然后通过组合消去 a_k 项从而得到更好的近似.
通常我们选取 $h_1=h$ 和 $h_2=rh$，其中 $r\in(0,1)$ (一般取 $r=\frac{1}{2}$):

$$\begin{cases} a=A(h)+\sum_{i=k}^{m}a_ih^i+c_m(h)h^{m+1} \\ a=A(rh)+\sum_{i=k}^{m}a_i(rh)^i+c_m(rh)(rh)^{m+1} \end{cases}$$

联立可得:

$$a-r^ka=A(rh)-r^kA(h)+\sum_{i=k+1}^{m}a_i(r^i-r^k)h^i+(c_m(rh)r^{m+1}-c_m(h)r^k)h^{m+1}$$

$$\Downarrow$$

$$a=\frac{A(rh)-r^kA(h)}{1-r^k}+\sum_{i=k+1}^{m}\tilde{a}_ih^i+\tilde{c}_m(h)h^{m+1}$$

$$\text{where } \begin{cases} \tilde{a}_i:=a_i(r^i-r^k)/(1-r^k) \quad (i=k+1,\dots,m) \\ \tilde{c}_m(h):=[c_m(rh)r^{m+1}-c_m(h)r^k]/(1-r^k) \end{cases}$$

如果我们将 $\tilde{A}(h):=[A(rh)-r^kA(h)]/(1-r^k)$ 视为 a 的近似，则误差逼近阶会由原来的 $O(h^k)$ 上升到 $O(h^{k+1})$
因此当 h 很小时，$\tilde{A}(h)$ 是比 $A(h)$ 更好的近似.

在 Richardson 外推中，我们只需知道 $A(h)$ 的逼近阶 k，
而不必知道系数 a_i $(i=k,\dots,m)$ 的值和函数 $c_m(h)$ 的解析式，
就可以得到逼近阶至少为 $k+1$ 的近似 $\tilde{A}(h):=[A(rh)-r^kA(h)]/(1-r^k)$
此外，如果系数 $\tilde{a}_{k+1}:=a_{k+1}(r^{k+1}-r^k)/(1-r^k)\neq 0$，
那么我们还能再进行 Richardson 外推得到更好的近似 (逼近阶至少是 $k+2$)，这个过程可以不断进行下去.
(值得注意的是，如果 $\tilde{a}_{k+1}=0$，则 $\tilde{A}(h)$ 的逼近阶至少是 $k+2$，此时进行 Richardson 外推可能会使精度降低)

具体来说，对于 $m\in\mathbb{N}$，定义 $A_{0,m}:=A(r^mh)$ 并构造:

$$A_{i+1,m}:=\frac{A_{i,m+1}-r^{k+i}A_{i,m}}{1-r^{k+i}}\quad(i=0,1,\dots)$$

这样我们就得到了如下的外推表:

表 5.3

$A_{0,0}$					
$A_{0,1}$	$A_{1,0}$				
$A_{0,2}$	$A_{1,1}$	$A_{2,0}$			
$A_{0,3}$	$A_{1,2}$	$A_{2,1}$	$A_{3,0}$		
$\vdots$	$\vdots$	$\vdots$	$\vdots$	$\ddots$	
$A_{0,n}$	$A_{1,n-1}$	$A_{2,n-2}$	$A_{3,n-3}$	$\cdots$	$A_{n,0}$

(2) Solution

设 f 在 x 附近充分光滑，则对于足够小的 $h>0$，我们有:

$$\begin{aligned}f(x+h)&=\sum_{k=0}^{\infty}\frac{f^{(k)}(x)}{k!}h^k\\&=f(x)+f'(x)h+\frac{f''(x)}{2!}h^2+\frac{f^{(3)}(x)}{3!}h^3+\cdots\\f(x-h)&=\sum_{k=0}^{\infty}\frac{f^{(k)}(x)}{k!}(-h)^k\\&=f(x)-f'(x)h+\frac{f''(x)}{2!}h^2-\frac{f^{(3)}(x)}{3!}h^3+\cdots\end{aligned}$$

于是我们有:

$$\begin{aligned}f(x+h)-f(x-h)&=2\sum_{k=1}^{\infty}\frac{f^{(2k-1)}(x)}{(2k-1)!}h^{2k-1}\\&=2f'(x)h+2\frac{f^{(3)}(x)}{3!}h^3+2\frac{f^{(5)}(x)}{5!}h^5+\cdots\end{aligned}$$

因此中点差分公式的误差表达式为:

$$\begin{aligned}f'(x)&=\frac{f(x+h)-f(x-h)}{2h}-\sum_{k=1}^{\infty}\frac{f^{(2k+1)}(x)}{(2k+1)!}h^{2k}\\&=A_0(h)-\frac{f^{(3)}(x)}{3!}h^2-\frac{f^{(5)}(x)}{5!}h^4-\cdots\end{aligned}$$

其中 $A_0(h):=\frac{f(x+h)-f(x-h)}{2h}$
利用 **Richardson 外推**对 h 逐次减半，则我们有:

$$A_m(h) := \frac{4^m A_{m-1}\left(\frac{h}{2}\right) - A_{m-1}(h)}{4^m - 1} \quad (m = 1, 2, \ldots)$$

外推过程如下:

$$\begin{array}{lllll}
A_0(h) & & & & \\
A_0\left(\frac{h}{2}\right) & \overset{①}{\rightarrow} & A_1(h) & & \\
A_0\left(\frac{h}{2^2}\right) & \overset{②}{\rightarrow} & A_1\left(\frac{h}{2}\right) & \overset{③}{\rightarrow} & A_2(h) \\
A_0\left(\frac{h}{2^3}\right) & \overset{④}{\rightarrow} & A_1\left(\frac{h}{2^2}\right) & \overset{⑤}{\rightarrow} & A_2\left(\frac{h}{2}\right) \quad \overset{⑥}{\rightarrow} \quad A_3(h) \\
\vdots & & \vdots & & \vdots \qquad\qquad \ddots
\end{array}$$

其截断误差满足:

$$f'(x) = G_m(h) + O(h^{2(m+1)})$$

考虑到舍入误差，一般 m 不能取太大.

Matlab 实现:

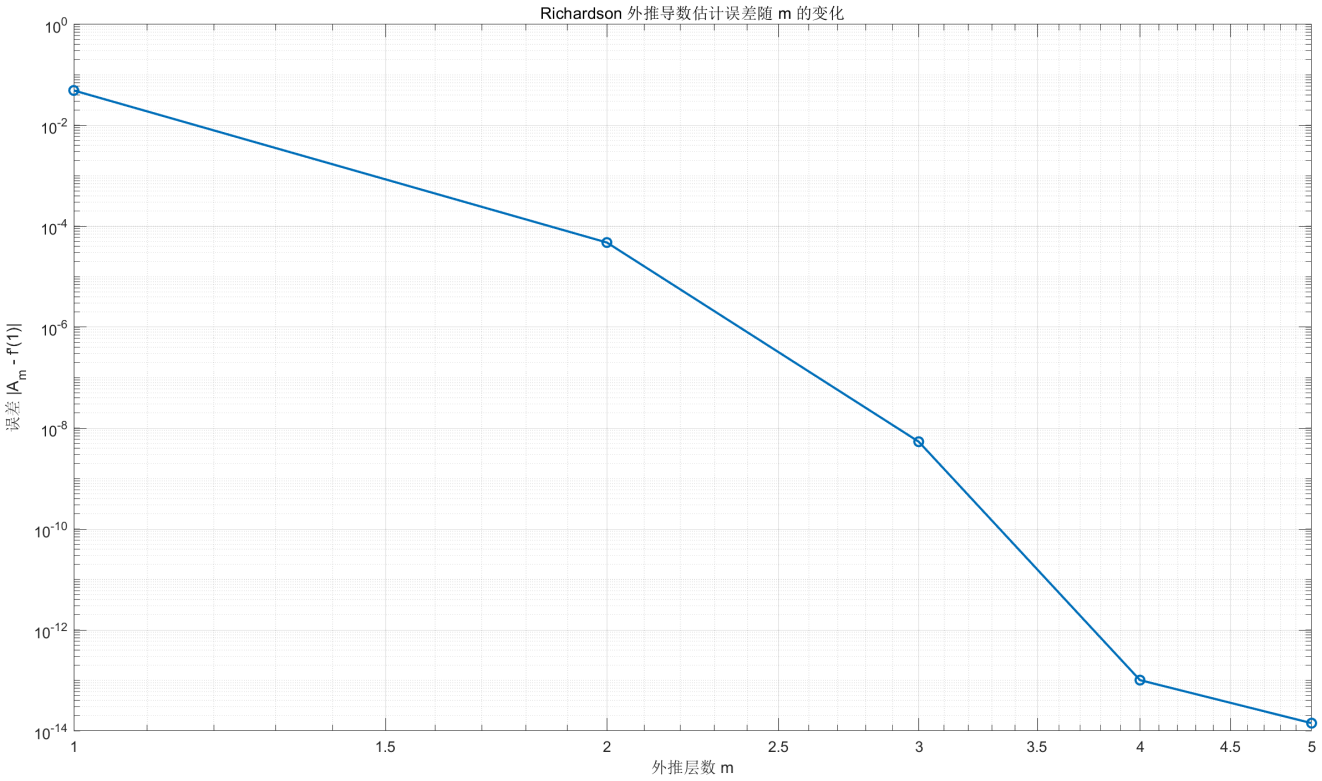
```
% 定义目标函数 f 及其精确导数 f'
f  = @(x) x.^3 .* exp(x);
df = @(x) exp(x) .* (3*x.^2 + x.^3);  % 精确导数：e^x (3x^2 + x^3)

x0 = 1;              % 估计点
exact = df(x0);      % 精确值 4e ≈ 10.8731
fprintf('精确导数 f''(1) = %.15f\n\n', exact);

% 外推参数
h0 = 0.5;            % 初始步长
M  = 8;              % 最大外推层数（A_m 最多 m=8）

% 初始化外推表 A，行 i 对应 h = h0/2^(i-1)，列 j 对应外推级数 j-1
A = NaN(M+1, M+1);

% 计算 A_0(h_i) = (f(x+h) - f(x-h)) / (2h)
for i = 1:(M+1)
    h = h0 / 2^(i-1);
    A(i,1) = ( f(x0 + h) - f(x0 - h) ) / (2*h);
end

% 递推填表：A(i,j) = (4^(j-1)*A(i,j-1) - A(i-1,j-1)) / (4^(j-1)-1)
for j = 2:(M+1)        % 外推层数 j-1 = 1..M
    for i = j:(M+1)
        A(i,j) = ( 4^(j-1)*A(i,j-1) - A(i-1,j-1) ) / (4^(j-1) - 1);
    end
end

% 显示外推表及误差
fprintf('  i    h_i           A(i,0)          A(i,1)          ...            A(i,%d)\n', M);
for i = 1:(M+1)
    h = h0 / 2^(i-1);
    fprintf('%3d  %10.3e', i, h);
    for j = 1:(M+1)
```

```
        if ~isnan(A(i,j))
            err = A(i,j) - exact;
            fprintf('    %+.5e', A(i,j), err);
        else
            fprintf('\t     -       ');
        end
    end
    fprintf('\n');
end

% 绘制最优误差随外推层数 m 的变化
errors = abs( diag(A, 0) - exact );  % 对角线 A(m,m)
ms = 0:M;
figure;
loglog(ms, errors, '-o', 'LineWidth',1.5);
xlabel('外推层数 m');
ylabel('误差 |A_m - f''(1)|');
title('Richardson 外推导数估计误差随 m 的变化');
grid on;
```

运行结果:

(邵老师: 程序的输出也应该是一个程序，或者至少是代码，就像下面这样)

i	h_i	A(i,0)	A(i,1)	A(i,2)	A(i,3)	A(i,4)	A(i,5)
0	5.000e-01	+1.49e+01 (+4.05e+00)	-	-	-	-	-
1	2.500e-01	+1.18e+01 (+9.75e-01)	+1.08e+01 (-4.91e-02)	-	-	-	-
2	1.250e-01	+1.11e+01 (+2.41e-01)	+1.09e+01 (-3.02e-03)	+1.09e+01 (+4.76e-05)	-	-	-
3	6.250e-02	+1.09e+01 (+6.02e-02)	+1.09e+01 (-1.88e-04)	+1.09e+01 (+7.38e-07)	+1.09e+01 (-5.37e-09)	-	-
4	3.125e-02	+1.09e+01 (+1.50e-02)	+1.09e+01 (-1.18e-05)	+1.09e+01 (+1.15e-08)	+1.09e+01 (-2.09e-11)	+1.09e+01 (+1.01e-13)	-
5	1.562e-02	+1.09e+01 (+3.76e-03)	+1.09e+01 (-7.35e-07)	+1.09e+01 (+1.80e-10)	+1.09e+01 (-9.59e-14)	+1.09e+01 (-1.42e-14)	+1.09e+01 (-1.42e-14)



Problem 4

Use a cubic spline function $s(x)$ to approximate $f(x) = e^x + \log(x)$ over $[1, 4]$.
Plot the first and second derivatives, as well as the approximation errors.
You are encouraged to try different step sizes
and observe the behavior of the error with respect to the step size.

Solution:
三次完全样条要求解的是这样一个稀疏线性方程组: **(可参考 Homework 4 Problem 4)**

$$\begin{bmatrix} \alpha_1 & \beta_1 & & & \\ \beta_1 & \alpha_2 & \beta_2 & & \\ & \beta_2 & \ddots & \ddots & \\ & & \ddots & \alpha_{n-2} & \beta_{n-2} \\ & & & \beta_{n-2} & \alpha_{n-1} \end{bmatrix} \begin{bmatrix} k_1 \\ k_2 \\ \vdots \\ k_{n-2} \\ k_{n-1} \end{bmatrix} = \begin{bmatrix} \eta_0 + \eta_1 - \beta_0 k_0 \\ \eta_1 + \eta_2 \\ \vdots \\ \eta_{n-3} + \eta_{n-2} \\ \eta_{n-2} + \eta_{n-1} - \beta_n k_n \end{bmatrix}$$

$$\text{where} \begin{cases} h_i = x_{i+1} - x_i & (i = 0, \ldots, n-1) \\ \beta_i = \frac{1}{h_i} & (i = 0, \ldots, n-1) \\ \alpha_{i+1} = 2(\beta_i + \beta_{i+1}) & (i = 0, \ldots, n-2) \\ \eta_i = 3\frac{y_{i+1} - y_i}{h_i^2} & (i = 0, \ldots, n-1) \end{cases}$$

其中 $x_0, x_1, \ldots, x_n$ 是插值节点.
三次样条函数及其一、二阶导数为:

$$
\begin{aligned}
s_i(x) &= y_i\phi\left(\frac{x_{i+1}-x}{h_i}\right)+y_{i+1}\phi\left(\frac{x-x_i}{h_i}\right)-k_ih_i\varphi\left(\frac{x_{i+1}-x}{h_i}\right)+k_{i+1}h_i\varphi\left(\frac{x-x_i}{h_i}\right)\\
s_i'(x) &= -\frac{y_i}{h_i}\phi'\left(\frac{x_{i+1}-x}{h_i}\right)+\frac{y_{i+1}}{h_i}\phi'\left(\frac{x-x_i}{h_i}\right)+k_i\varphi'\left(\frac{x_{i+1}-x}{h_i}\right)+k_{i+1}\varphi'\left(\frac{x-x_i}{h_i}\right)\\
s_i''(x) &= \frac{y_i}{h_i^2}\phi''\left(\frac{x_{i+1}-x}{h_i}\right)+\frac{y_{i+1}}{h_i^2}\phi''\left(\frac{x-x_i}{h_i}\right)-\frac{k_i}{h_i}\varphi''\left(\frac{x_{i+1}-x}{h_i}\right)+\frac{k_{i+1}}{h_i}\varphi''\left(\frac{x-x_i}{h_i}\right)
\end{aligned}
\quad (i=0,1,\dots,n-1)
$$

$$
\begin{aligned}
\text{where } \phi(x) &= 3x^2-2x^3,\ \varphi(x)=x^3-x^2\\
\phi'(x) &= 6x-6x^2,\ \varphi'(x)=3x^2-2x\\
\phi''(x) &= 6-12x,\ \varphi''(x)=6x-2
\end{aligned}
$$

在等距节点的假设下，部分变量的形式可以得到简化:

$$
\begin{aligned}
x_{i+1}-x_i &\equiv h\\
\beta &= \frac{1}{h}\\
\alpha &= 2(\beta+\beta)=4\beta
\end{aligned}
$$

于是上述稀疏线性方程组变为:

$$
\begin{bmatrix}
4\beta & \beta & & & \\
\beta & 4\beta & \beta & & \\
 & \beta & \ddots & \ddots & \\
 & & \ddots & 4\beta & \beta\\
 & & & \beta & 4\beta
\end{bmatrix}
\begin{bmatrix}
k_1\\ k_2\\ \vdots\\ k_{n-2}\\ k_{n-1}
\end{bmatrix}
=
\begin{bmatrix}
\eta_0+\eta_1-\beta k_0\\ \eta_1+\eta_2\\ \vdots\\ \eta_{n-3}+\eta_{n-2}\\ \eta_{n-2}+\eta_{n-1}-\beta k_n
\end{bmatrix}
$$

$$
\text{where } \beta=\frac{1}{h} \text{ and } \eta_i=3\frac{y_{i+1}-y_i}{h^2}\ (i=0,\dots,n-1)
$$

Matlab 代码:

```
function [x_ex, s, sp, spp, f_ex, fp_ex, fpp_ex, err_s, err_sp, err_spp] = ...
         Complete_Cubic_Spline_Derivs( f, f1, f2, n, k1, kn, interval )
    % f, f1, f2    : function handles for f, f', f''
    % n            : number of nodes
    % k1, kn       : clamped boundary derivatives
    % interval     : [a, b]

    a = interval(1);  b = interval(2);
    x_ex = linspace(a, b, 1000);
    h = (b - a)/(n-1);
    x_nodes = linspace(a, b, n);
    y_nodes = f(x_nodes);

    % --- solve for k(2:n-1) as before ---
    beta = 1/h;
    n_int = n-2;
    A = diag(4*beta*ones(n_int,1)) ...
      + diag(beta*ones(n_int-1,1),1) ...
      + diag(beta*ones(n_int-1,1),-1);
    eta = 3*(y_nodes(2:end)-y_nodes(1:end-1))/h^2;
    bvec = eta(1:end-1)+eta(2:end);
    bvec(1)   = bvec(1)   - beta*k1;
    bvec(end) = bvec(end) - beta*kn;
    k_int = A \ bvec';
    k = [k1; k_int; kn];
```

```
% --- preallocate ---
s   = zeros(size(x_ex));
sp  = zeros(size(x_ex));
spp = zeros(size(x_ex));

% shape funcs and derivatives
phi    = @(u)   3*u.^2  - 2*u.^3;
phip   = @(u)   6*u     - 6*u.^2;
phipp  = @(u)   6       - 12*u;
varphi = @(u)   u.^3    - u.^2;
varp   = @(u)   3*u.^2 - 2*u;
varpp  = @(u)   6*u     - 2;

% --- loop over evaluation points ---
for j = 1:length(x_ex)
    x = x_ex(j);
    i = min( floor((x - a)/h)+1, n-1 );    % segment index
    u = (x - x_nodes(i))/h;
    v = 1 - u;
    yi = y_nodes(i);    yi1 = y_nodes(i+1);
    ki = k(i);          kip1 = k(i+1);

    % spline value
    s(j) = yi*phi(v) + yi1*phi(u) - h*ki*varphi(v) + h*kip1*varphi(u);

    % first derivative
    sp(j)= -yi*(phip(v)/h) + yi1*(phip(u)/h) + ki*varp(v) + kip1*varp(u);

    % second derivative
    spp(j)=  yi*(phipp(v)/h^2) + yi1*(phipp(u)/h^2) - ki*(varpp(v)/h) + kip1*(varpp(u)/h);
end

% exact
f_ex   = f(x_ex);
fp_ex  = f1(x_ex);
fpp_ex = f2(x_ex);

% errors
err_s   = abs(s   - f_ex);
err_sp  = abs(sp  - fp_ex);
err_spp = abs(spp - fpp_ex);

% ---- PLOTTING ----
figure;

% 1) function and error
subplot(3,2,1);
plot(x_ex,f_ex,'k-','LineWidth',1.5); hold on;
plot(x_ex,s,'r--'); scatter(x_nodes,y_nodes,15,'k','filled');
title('f vs s'); legend('f','s','nodes'); grid on;

subplot(3,2,2);
plot(x_ex,log(err_s),'b-.','LineWidth',1);
title('log|s - f|'); grid on;

% 2) first derivative
```

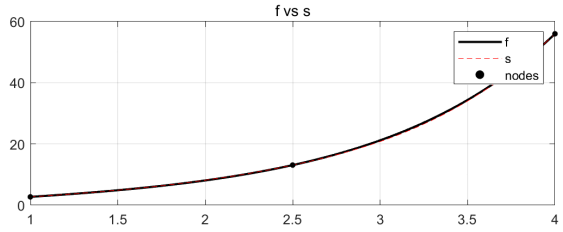
```
    subplot(3,2,3);
    plot(x_ex,fp_ex,'k-','LineWidth',1.5); hold on;
    plot(x_ex,sp,'r--'); scatter(x_nodes,f1(x_nodes),15,'k','filled');
    title('f'' vs s'''); legend('f'''','s'''','nodes'); grid on;

    subplot(3,2,4);
    plot(x_ex,log(err_sp),'b-.','LineWidth',1);
    title('log|s'' - f''|'); grid on;

    % 3) second derivative
    subplot(3,2,5);
    plot(x_ex,fpp_ex,'k-','LineWidth',1.5); hold on;
    plot(x_ex,spp,'r--'); scatter(x_nodes,f2(x_nodes),15,'k','filled');
    title('f'''' vs s'''''); legend('f'''''','s'''''','nodes'); grid on;

    subplot(3,2,6);
    plot(x_ex,log(err_spp),'b-.','LineWidth',1);
    title('log|s'''' - f''''|'); grid on;
end
```

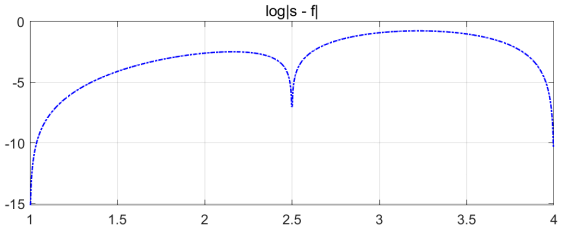
函数调用:

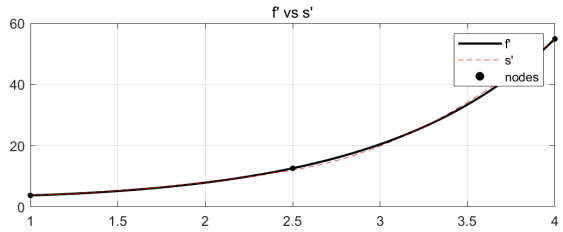
```
f  = @(x) exp(x) + log(x);
f1 = @(x) exp(x) + 1./x;
f2 = @(x) exp(x) - 1./x.^2;

n   = 3;       % try 3, 4, 5, …
k1  = f1(1);   % clamped at x=1
kn  = f1(4);   % clamped at x=4
[a,xs,sp,spp,fe,fpe,fppe,es,esp,espp] = Complete_Cubic_Spline_Derivs(f,f1,f2,n,k1,kn,[1,4]);
```

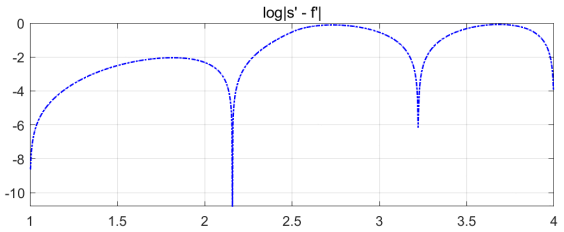
运行结果:

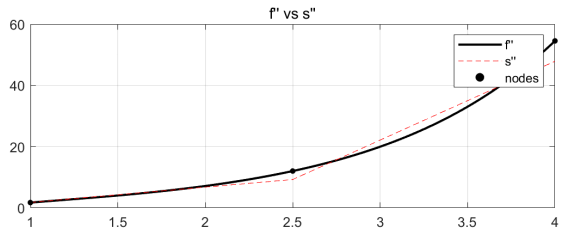
- $n = 3$ **的情况:**

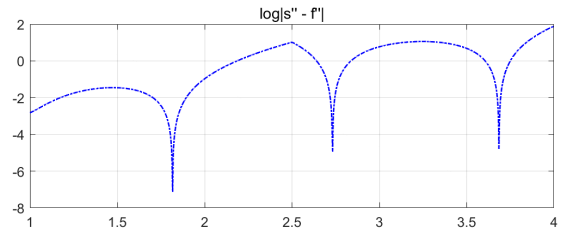




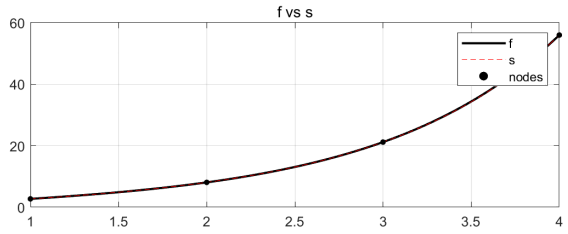


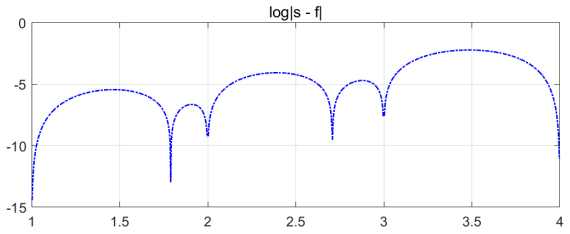


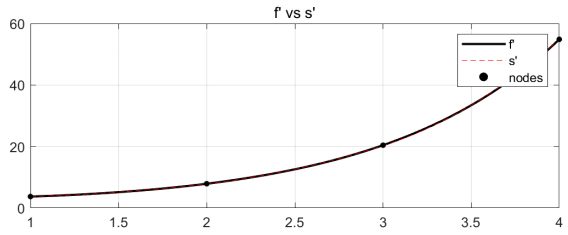


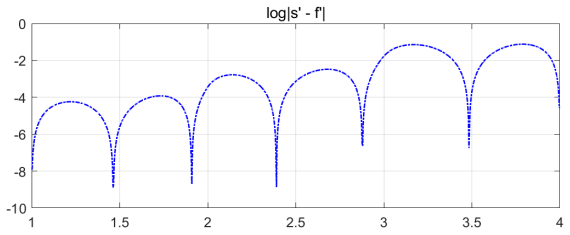


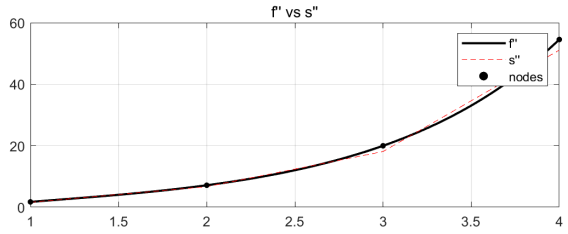
- $n = 4$ **的情况:**

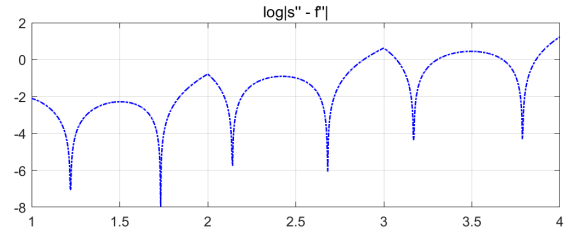




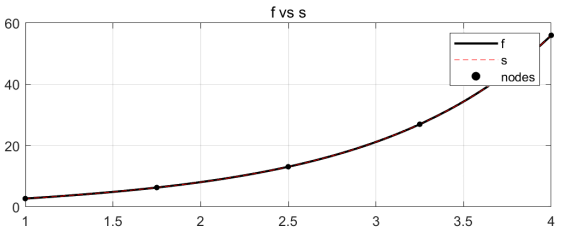


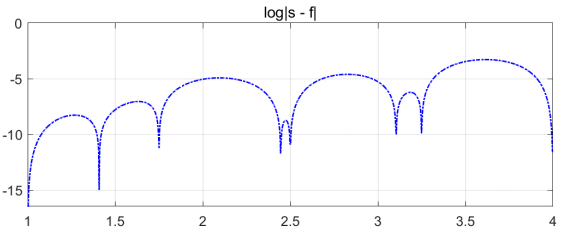


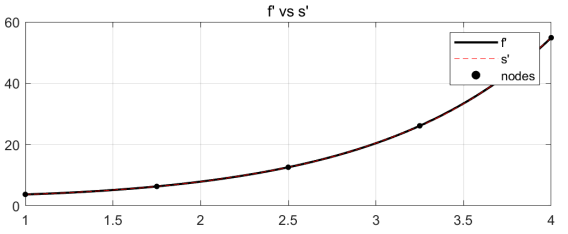


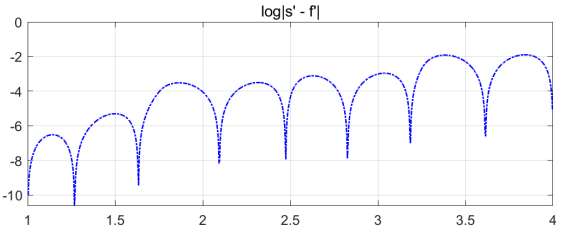


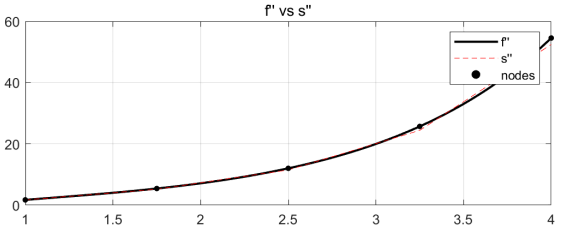
- $n = 5$ **的情况:**

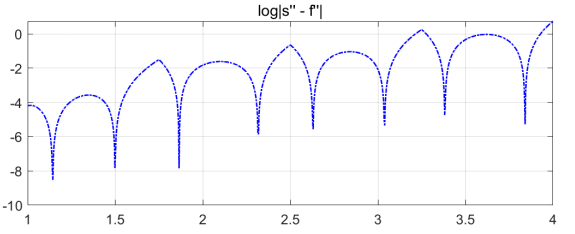












(optional)

Use contour integration to compute the first and second derivatives.

Problem 5 (optional)

An ancient way of computing π can be interpreted in modern terms as $\pi \approx n\sin(\pi/n)$,
where n is typically chosen as $n = 3\cdot 2^k$ for $k \in \mathbb{N}$.
Unfortunately, the convergence of such an approximation is very slow,
and the calculation is expensive——it involves a lot of square roots
(because ancient mathematicians had to use geometry instead of Taylor series to compute $\sin(\pi/n)$).
Some scholars believe that the Chinese mathematician Zu Chongzhi
discovered a way to largely accelerate the calculation using some sort of extrapolation.

Use Richardson extrapolation to calculate π to seven correct digits after the decimal point,
based on the asymptotic expansion:

$$n\sin(\frac{\pi}{n}) = \pi - \frac{\pi^3}{3!n^2} + \frac{\pi^5}{5!n^4} - \cdots$$

What is the largest value of n in your calculation?
For simplicity, you may call library functions
to perform calculations such as `sin(pi/n)` directly in your program.

解: 记 $F_2(n) = \pi - \frac{\pi^3}{3!n^2} + \frac{\pi^5}{5!n^4} \cdots$

$$= \pi + k_2\cdot\frac{1}{n^2} + k_4\cdot\frac{1}{n^4} + \cdots$$

那么, 可构造:

$$F_4(n) = \frac{2^2 F_2(2n) - F_2(n)}{2^2 - 1}$$

$$= \pi + \tilde{k}_4\cdot\frac{1}{n^4} + \cdots$$

一般的:

$$F_{2k}(n) = \frac{2^{2(k-1)} F_{2(k-1)}\cdot(2n) - F_{2(k-1)}(n)}{2^{2(k-1)} - 1}$$

从 $n_0 = 3$ 开始迭代, 到达 F_8 时, 也即 n 最大为 24 时已经计算得到 7 位正确数字:

```
     3      6     12     24

inter_results =

   2.598076211353316   3.133974596215561   3.141580063335317   3.141592650573243
```

□

Problem 6 (optional)

Read the paper "Inverse Problems Light: Numerical Differentiation"
by M. Hanke and O. Scherzer in the reading folder.
Implement the numerical differentiation algorithm in this paper.

Problem 7 (optional)

5

5. Try to find constants c_1, c_2, and c_3 such that the degree of exactness of the following quadrature rule is maximized:

$$\int_{-2a}^{2a} f(x)\,\mathrm{d}x \approx c_1 f(-a) + c_2 f(0) + c_3 f(a).$$

解: $c_1 = c_3 = \frac{8}{3}a$, $c_2 = -\frac{4}{3}a$, 代数精度为3

以下是证明:

对于 $f(x) = 1$ 有:

$$\int_{-2a}^{2a} f(x)\,dx = \int_{-2a}^{2a} 1 \cdot dx = 4a$$

$$c_1 f(-a) + c_2 \cdot f(0) + c_3 f(a)$$
$$= \frac{8}{3}a - \frac{4}{3}a + \frac{8}{3}a$$
$$= 2a$$

故 $f(x) = 1$ 时 有等号成立

$f(x) = x$ 时.

$$\int_{-2a}^{2a} f(x)\,dx = \int_{-2a}^{2a} x\,dx = 0$$

$$c_1 f(-a) + c_2 \cdot f(0) + c_3 f(a)$$
$$= \frac{8}{3} \cdot (-a) - \frac{4}{3}a \cdot 0 + \frac{8}{3}a$$
$$= 0$$

故 $f(x) = x$ 时等号成立

$f(x) = x^2$ 时.

$$\int_{-2a}^{2a} f(x)\,dx = \int_{-2a}^{2a} x^2\,dx = \frac{1}{3}x^3 \Big|_{-2a}^{2a} = \frac{16}{3}a^3$$

$$C_1 f(-a) + C_2 \cdot f(0) + C_3 f(a)$$
$$= \frac{5}{3}a \cdot a^2 + \frac{4}{3}a \cdot 0 + \frac{5}{3}a \cdot a^2$$
$$= \frac{16}{3}a^3$$

故 $f(x) = x^2$ 时也满足

$f(x) = x^3$ 时.

$$\int_{-2a}^{2a} f(x)\,dx = \int_{-2a}^{2a} x^3\,dx = 0$$

$$C_1 f(-a) + C_2 \cdot f(0) + C_3 f(a)$$
$$= \frac{5}{3}a \cdot (-a^3) + \frac{8}{3} \cdot a \cdot 0 + \frac{5}{3}a \cdot a^3$$
$$= 0$$

故 $f(x) = x^3$ 也成立

再考虑 $f(x) = x^4$ 时

$$\int_{-2a}^{2a} x^4\,dx = \frac{1}{5}x^5 \Big|_{-2a}^{2a} = \frac{64}{5}a^5$$

$$C_1 f(-a) + C_2 \cdot f(0) + C_3 f(a) = \frac{16}{3}a^5$$

等不满足！

因此 $C_1 = C_3 = \frac{5}{3}a$, $C_2 = -\frac{4}{3}a$ 时，代数精度为 3. □

Problem 8 (optional)

Solution:

对于连续函数来说，Gauss 型求积公式在 $n \to \infty$ 时收敛于积分的精确值.

给定 $[a, b]$ 上的连续函数 f，考虑积分 $I[f] = \int_a^b \rho(x) f(x) \mathrm{d}x$

记 n 节点的 Gauss 型求积公式为 $I_n^G[f] := \sum_{k=1}^n A_k f(x_k)$

> **(Weierstrass 第一定理, 数值逼近, 定理 4.1)**
> 任意给定 $f \in C([a,b])$ 和 $\varepsilon > 0$
> 都存在代数多项式 $p(x)$ 满足 $\|f-p\|_\infty < \varepsilon$
> 换言之，任意给定 $f \in C([a,b])$，
> 都存在多项式序列 $\{p_n\}_{n=1}^{\infty}$ 一致收敛 (即依范数 $\|\cdot\|_\infty$ 收敛) 于 f:
>
> $$\lim_{n\to\infty} \|f-p_n\|_\infty = 0$$

根据 Weierstrass 第一定理可知:
对于任意 $\varepsilon > 0$，都存在正整数 N，
使得对于任意 $n > N$ 都存在 n 次多项式函数 $p_n(x)$ 满足 $\|f-p_n\|_\infty < \varepsilon$.
于是我们有:

$$
\begin{aligned}
|I[f]-I_n^G[f]| &= |I[f]-I[p_n]+I[p_n]-I_n^G[p_n]+I_n^G[p_n]-I_n^G[f]| \\
&\le |I[f]-I[p_n]|+|I[p_n]-I_n^G[p_n]|+|I_n^G[p_n]-I_n^G[f]| \\
&\le (b-a)\max_{x\in[a,b]}|f(x)-p_n(x)|+0+\sum_{k=1}^{n}|A_k|\cdot|p_n(x_k)-f(x_k)| \\
&= (b-a)\max_{x\in[a,b]}|f(x)-p_n(x)|+\sum_{k=1}^{n}|A_k|\cdot\max_{x\in[a,b]}|f(x)-p_n(x)| \\
&= \left(b-a+\sum_{k=1}^{n}|A_k|\right)\|f-p_n\|_\infty \\
&< \left(b-a+\sum_{k=1}^{n}|A_k|\right)\varepsilon
\end{aligned}
$$

其中 $|I[p_n]-I_n^G[p_n]|=0$ 是因为 n 节点的 Gauss 型求积公式具有 $2n-1$ 次代数精度，
因此对于 n 次多项式函数 $p_n(x)$ 来说是精确的:

$$
I[p_n]=\int_a^b\rho(x)p_n(x)=\sum_{k=1}^{n}A_kp_n(x_k)=I_n^G[p_n]
$$

尽管上述证明针对的是积分区间为有限区间 $[a,b]$ 的情况，
但当积分区间无界时也有类似的结论，不过由于 Weierstrass 第一定理不再成立，会有些麻烦，
一种处理方法是将依 $\|\cdot\|_\infty$ 的收敛 (一致收敛) 放宽到依 $\|\cdot\|_2$ 的收敛.

4

4. Let $E_n[f]$ be the remainder of the n-point Gauss–Legendre quadrature rule. Show that for any continuous function $f \in C[-1,1]$, we have

$$\lim_{n\to\infty} E_n[f] = 0,$$

i.e., Gauss–Legendre quadrature rule converges for any continuous function.

解：记 n 点 Gauss-Legendre 积分公式为 I_n，即：

$$I_n[f] = \sum_{k=1}^{n} A_k f(x_k)$$

其中 x_k 为 n 阶 Legendre 多项式的零点

再设 $\deg p^* \le n$ 并且 p^* 是 $f(x) \in C[-1,1]$ 的最佳逼近多项式

那么：

$$|I[f] - I_n[f]|$$

$$= |I[f] - I[p^*] + I[p^*] - I_n[f]|$$

$$= |I[f] - I[p^*] + I_n[p^*] - I_n[f]|$$

$$\le |I[f] - I[p^*]| + |I_n[p^*] - I_n[f]|$$

$$= \left|\int_{-1}^{1} (f(x) - p^*(x))\,dx\right| + \left|\sum_{k=1}^{n} A_k \cdot (p^*(x_k) - f^*(x_k))\right|$$

$$\leq 2 \cdot \|f(x)-p^*(x)\|_\infty + \|f(x)-p^*(x)\|_\infty \cdot \sum_{k=1}^{n} |A_k|$$

$$\leq \|f(x)-p^*(x)\|_\infty \left(2 + \sum_{k=1}^{n} |A_k|\right) \quad (*)$$

设 $l_k(x) = \dfrac{(x-x_1)\cdots(x-x_{k-1})\cdot(x-x_{k+1})\cdots(x-x_n)}{(x_k-x_1)\cdots(x_k-x_{k-1})\cdot(x_k-x_{k+1})\cdots(x_k-x_n)}$

$\deg l_k^2(x) = 2n-2 \leq 2n-1$

则 Gauss-Legendre 公式对 $l_k^2(x)$ 精确成立

$$\int_{-1}^{+1} l_k^2(x)\,dx = \sum_{i=1}^{n} A_i\, l_k^2(x_i) = A_i > 0$$

注意到对于 $g(x)=1$ $\deg g(x) = 0 \leq 2n-1$

则 $\int_{-1}^{+1} g(x)\,dx = \sum_{i=1}^{n} A_k \cdot 1 = 2$

$$\Rightarrow \sum_{i=1}^{n} |A_k| = \sum_{i=1}^{n} A_k = 2$$

则由 (*) 式：

$$|I[f]-I_n[f]| \leq 4 \cdot \|f(x)-p^*(x)\|_\infty$$

由于 p^* 是 $\deg p^* \leq n$ 的最佳逼近多项式

由 Weierstrass 定理，$n \to +\infty$ 时，$\|f(x) - p^*(x)\|_\infty \to 0$

也即 $\lim\limits_{n \to +\infty} E_n[f] = \lim\limits_{n \to +\infty} |I[f] - I_n[f]| = 0$

□